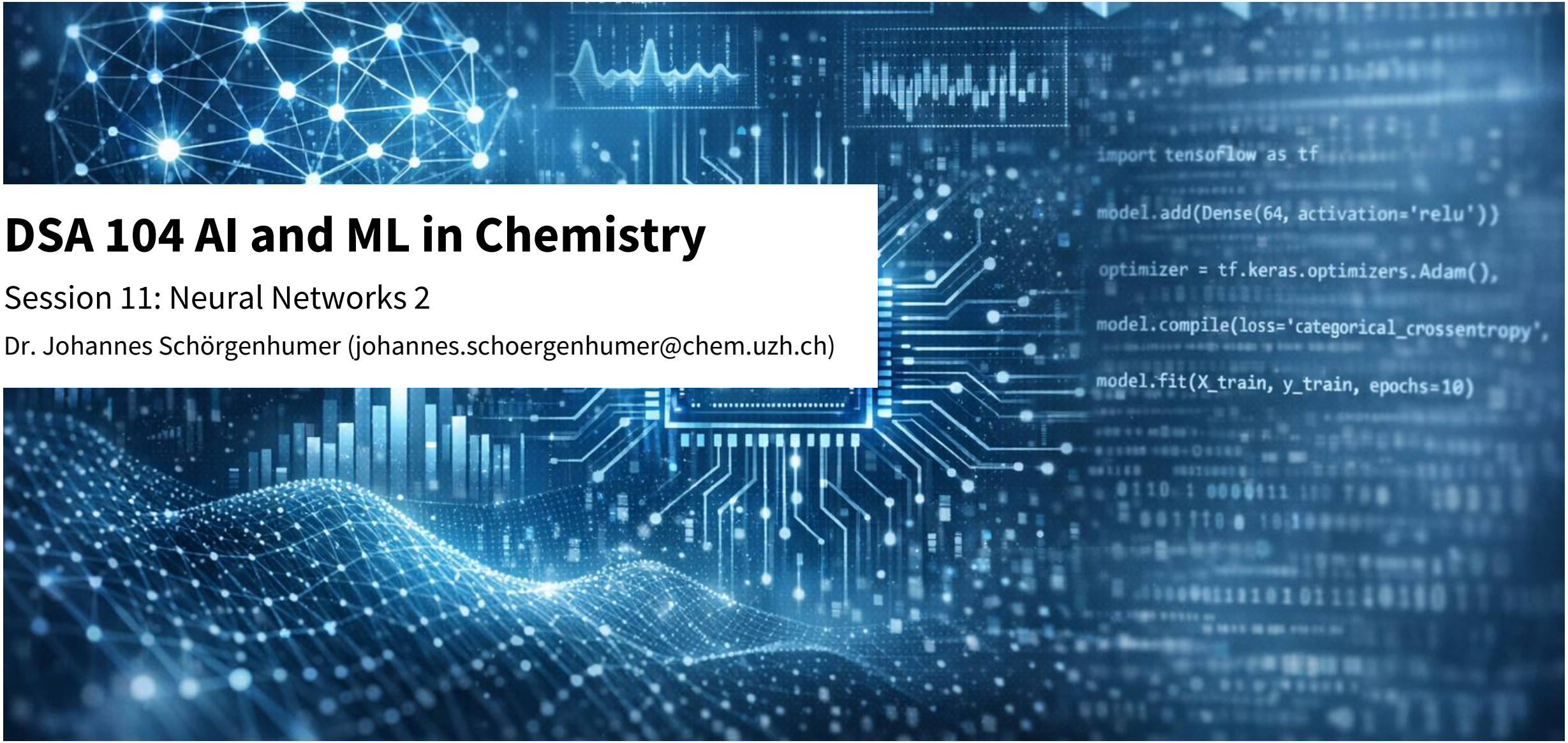
Universität Zürich[UZH]

Department of Chemistry



DSA 104 AI and ML in Chemistry

Session 11: Neural Networks 2

Dr. Johannes Schörgenhumer (johannes.schoergenhumer@chem.uzh.ch)

PyTorch Workshop

1) Optimise the NN for the diabetes dataset!

Investigate in pairs / small groups:

— Width and depth of the NN

— Epochs

— Learning_rate

— Loss functions

— Optimisers (e.g. compare training time)

— Compare to the results of the linear regressions

What happens if you simply run the train the model again without reinitialising the model (rerunning the cell where the model is first called)?

2) Batch training

Applicability of neural networks

NN very powerful, **but not suited for every problem**!

Neural networks perform best when:

— Dataset is **large**

— Relationships are **complex**

— The problem is **non-linear** and **high-dimensional**

— **Interpretability** is less important

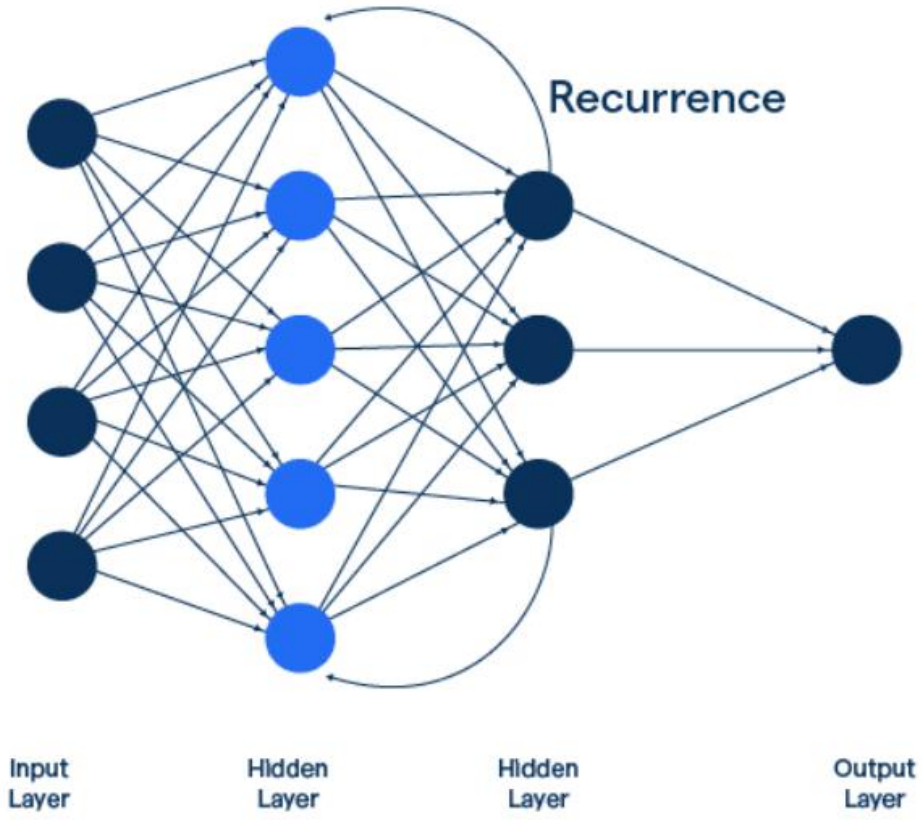
— You have **enough computing resources**

E.g. Language, speech, image processing, machine vision, generative models, …

For small datasets, classical ML often works **better**

Types of Neural Networks

- **FNN/MLP (Feedforward NN)**
- **CNN (Convolutional NN)**
- RNN (Recurrent NN)
- Transformers
- Encoders
- GAN (Generative Adversarial N)
- **GNN (Graph NN)**
- RL networks (Reinforcement Learning N)
- Diffusion models



https://medium.com/%40sumbatilinda/deep-learning-part4-recurrent-neural-network-rnn-0714e0852581

Learning Molecular Structure: An excursion to RNNs and Transformers

- Encoding molecular structure from SMILES:
  - Fingerprints
  - One-hot encoding (substructures, functional groups)
  - Molecular descriptors

- What if we can skip the corner and use the strings directly?

- **Learning from text: What makes SMILES strings more difficult than regular text?**
  - Rings, branches: long dependencies, nested structures
  - Syntax constraints
  - Non-uniqueness

Different approaches: **RNNs** and **Transformers**

Learning from text: RNNs

- Processes **sequences** step-by-step
- Every neuron maintains hidden state ("**memory**") of previous inputs $h_t = \tanh(Wx_t + Uh_{t-1} + b)$
- Allows to learn patterns over longer sequences
- Perfect for everything where **order matters**:
  - Timeseries
  - Text
  - Audio
- Pure RNN models (e.g. Backpropagation through time (BPTT)) outdated

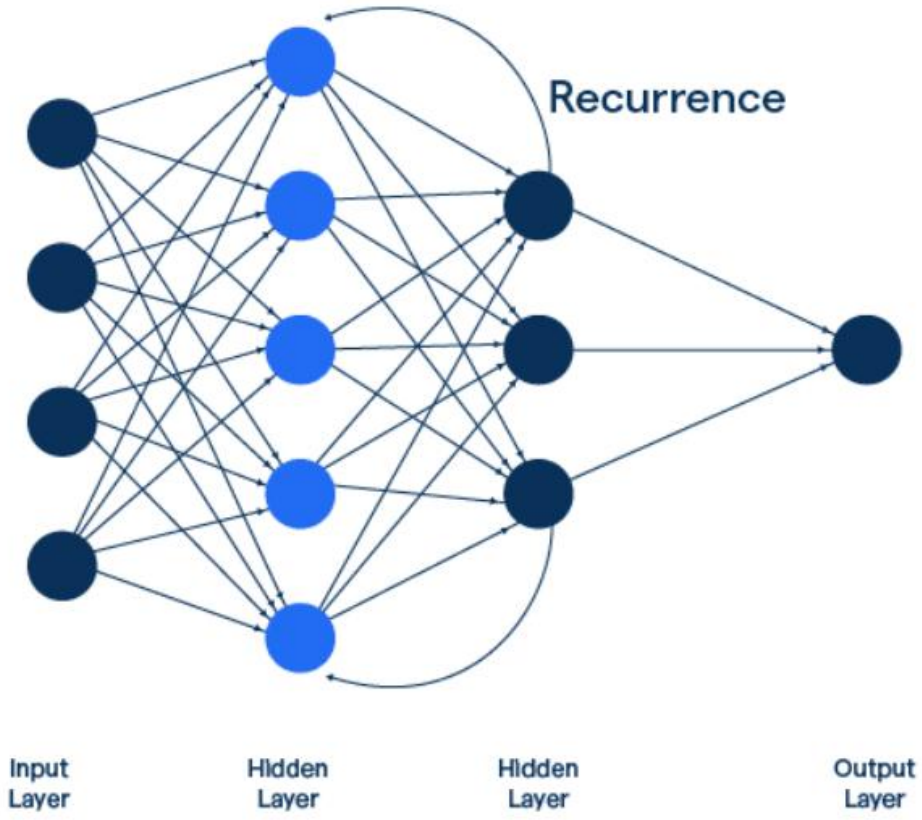
- Processes **sequences** step-by-step
- Every neuron maintains hidden state ("**memory**") of previous inputs $h_t = \tanh(Wx_t + Uh_{t-1} + b)$
- Allows to learn patterns over longer sequences
- Perfect for everything where **order matters**:
  - Timeseries
  - Text
  - Audio
- Pure RNN models (e.g. Backpropagation through time (BPTT)) outdated

https://medium.com/%40sumbatilinda/deep-learning-part4-recurrent-neural-network-rnn-0714e0852581

How RNNs learn:

- Process sequence, e.g. SMILES:
  - c1ccccc1: c → 1 → c → c → c → c → c → 1
- At each step:
  - Input token
  - Update hidden state
  - Predict next token / property

- RNN learns:
  - Common substructures (e.g. C=O)
  - Syntax rules (e.g. opening/closing brackets)



https://medium.com/%40sumbatilinda/deep-learning-part4-recurrent-neural-network-rnn-0714e0852581

Pros and cons of RNNs:

Pros:

— Simple

— Works with small data

— Sequential modeling

Cons:

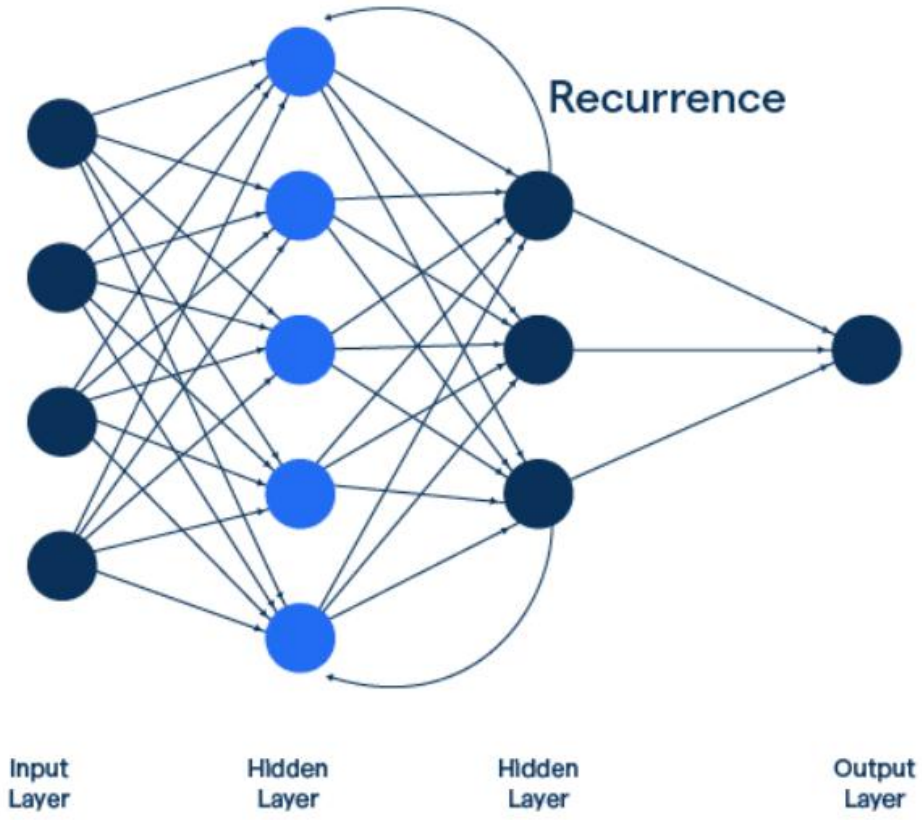
— Weak long-range **memory** (vanishing gradients)

  — Early tokens fade from memory

  — Hard to match long branches or rings…

— Slow training

— Error accumulation

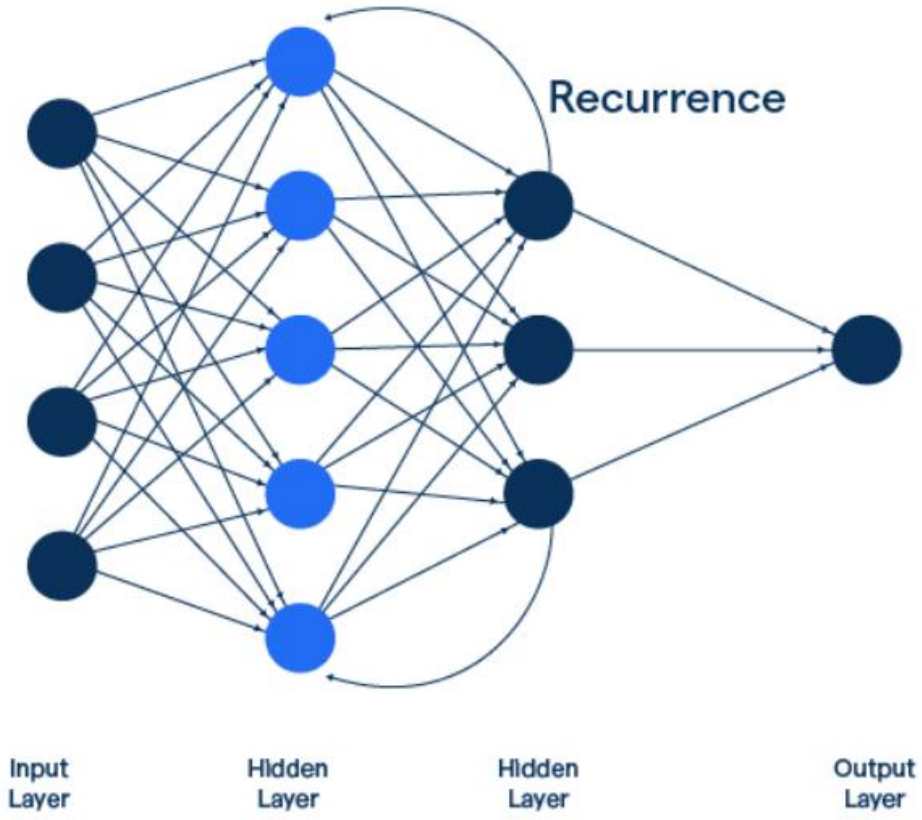


https://medium.com/%40sumbatilinda/deep-learning-part4-recurrent-neural-network-rnn-0714e0852581

Evolution and application of RNNs:

- Long Short-Term Memory (LSTM):
  - Added gates do decide which memories to keep, update and forget
  - Much better with long-range dependencies
- Gated Recurrent Unit (GRU):
  - Essentially like LSTMs but with fewer gates and much faster

- RNN not too important anymore:
  - Outdated and **outperformed by transformers**
  - Still matter for understanding sequence modeling
  - Relevant in some light-weight applications



https://medium.com/%40sumbatilinda/deep-learning-part4-recurrent-neural-network-rnn-0714e0852581

Tokenisation

Tokens = meaningful symbols / discrete units that a model understands

Why? Letter by letter does not work, e.g. in

— 3-Letter AA: “Ala” → “A”, “l”, “a” ✗

— SMILES: “Br” → “B”, “r” ✗

Not trivial! Consider:

— Brackets, e.g. [NH4+], [Si]

— Special symbols for bonds, stereocentres, …

Different tokenisers, e.g. Regex systems, sub-word, word-level

NNs need numbers:

1) Encode tokens as integers
2) Apply **padding** for equal length (“dummy tokens”)
3) Encode integers in one-hot vectors

Encoders & Decoders

Encoder: transforms raw **input into a meaningful latent representation** (feature vector, embedding)

— Extracts relevant features

— Removes noise / redundancy

— Often many-to-one (e.g., sequence → vector)

Examples:

— NLP: Turns a sentence into contextual embeddings

— Vision: Converts an image into a feature map

— Autoencoders: Compress data into a low-dimensional bottleneck

Decoder: takes the latent representation and **reconstructs or generates output**

— Reconstructs, generates, or predicts

— Often one-to-many (vector → sequence)

Examples:

— Generate a translated sentence

— Reconstruct image from latent code

— Predict future tokens in a Transformer decoder (generate SMILES, AA sequences)

Common architectures:

— Seq2Seq (RNN based)

— Autoencoders

— Transformer-based models (e.g. GPT = decoder only)

RNN Exercise

Run the *pytorch_RNN* notebook.

The model will generate some smiles codes based on the grammar it learned from the training data.

- First try on the toy dataset
- Rerun on a sample of the ESOL dataset
- Note that the output is set to a fixed length, try a few different values, collect some smiles codes
- Inspect the outputs for the different datasets and discuss with your neighbours.
- What issues do you observe?

Transformers

Deal with memory issue of RNNs…

Attention instead of recurrence:

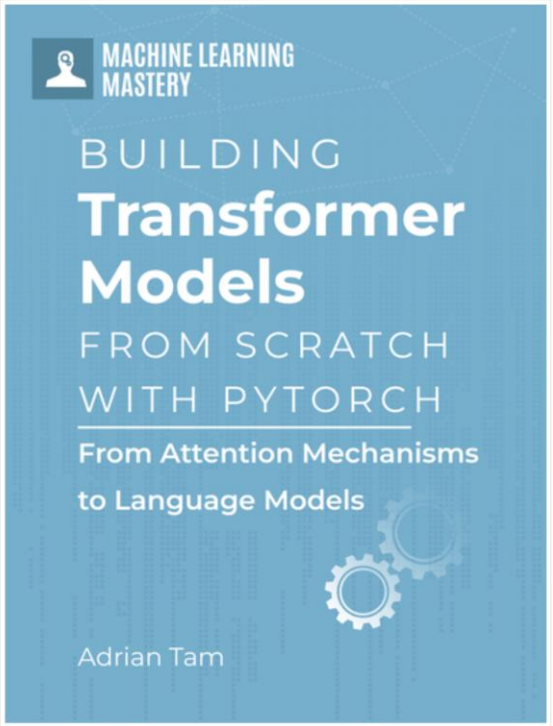
— **Self-attention** calculates weights between all tokens to determine their relevance

— Processes entire sequence **at once, not sequential** (RNNs!)

DL model introduced 2017 by Google

— RNNs and LSTM mostly redundant now

— Paved way for models like GPT



Vaswani et al, “Attention is all you need”, **2017**, https://doi.org/10.48550/arXiv.1706.03762

Transformers: Working principle

- Input tokens → embeddings
  - The characters/tokens (letters in SMILES) become vectors
- Add positional encoding
  - No recurrence, explicit information about order needed
- **Self-attention** layers, each layer computes (multiple sets of):
  - Query (Q), Key (K), Value (V)
  - Attention scores = softmax($Q \cdot K^T$)
  - Output = attention-weighted combination of V
- Feed-forward **networks process** each token (**including contextual information**) independently / in parallel
- Stack many layers each with **multi-head self attention** (MHSA): rich contextual representations
- Decoder (optional) generates sequences token-by-token using masked attention

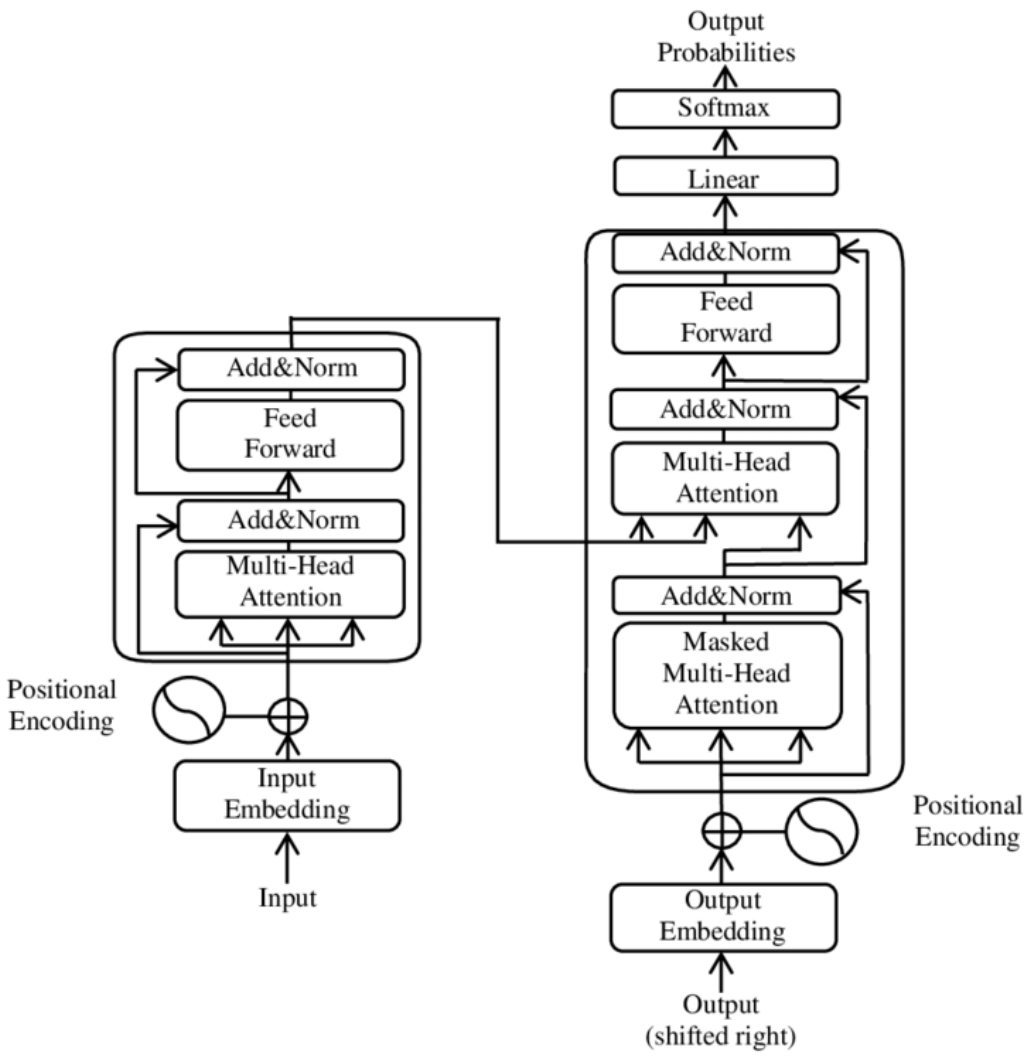


Image by Yuening Jia - DOI:10.1088/1742-6596/1314/1/012186, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=121340680

Transformers – How attention works for SMILES:

Self-supervised learning of chemistry:

— Objective: try to predict next token.

e.g. c1ccccc1O

— Q for last “1”: Which atoms starts my ring?

— K for first “1”: I am a ring opening marker.

— Dot-product of Q and K high (strong attention)

— V passed from “1” to the other “1”

Model learns:

— where the ring began

— which atoms form the ring

— how substitution affects aromaticity

Different attention heads for different sub-problems

Key idea: Model understands **global structure instead of only neighbours**

Typical pipeline:

1) Tokenisation
2) Embedding + Positional encoding
3) Encoder for property prediction (QSAR), reaction classification, retrosynthesis prediction, …
4) Decoder (optional): molecule generation, reaction product generation
5) Sampling strategies:
   — greedy (highest probability)
   — beam search (focus on best partial sequences)
   — top-k sampling (take k most probable, renormalise probabilities and sample from there)
   — nucleus sampling (cumulated probability > x)
   — random sampling

Pros and cons of Transformers:

Pros:

— Parallel computing! Scale massively with GPUs

— Learn long-range relationships easily

— Train faster and more effectively

— Produce richer representations than RNNs

Cons:

— Data-hungry

— Computationally expensive (energy!)

— Less intuitive

https://medium.com/%40sumbatilinda/deep-learning-part4-recurrent-neural-network-rnn-0714e0852581

RNNs vs. Transformers

Feature	RNN	Transformer
Processing	Sequential	Parallel
Memory	Hidden state	Attention
Long-range deps	Weak	Strong

When to use what:

Scenario	Model
Small dataset	RNN
Large dataset	Transformer
Complex structures	Transformer

Transformer Demo

Have a look at the difference to Smiles codes generated by a transformer.

Note that a transformer could do much more, as soon as it understands the grammar, e.g. train it for solubility prediction.